# Python Course 2025/26/2

## Lecture 4: Tuples, Dictionaries, Functions

Course coordinator: Dr Mate Tejfel

Lecturer: Zoltan Kegli

# Lecture Overview

## Topics

- Quick recap: lists and mutability
- Tuples and immutability
- Dictionaries and key-value modeling
- Sets and membership operations
- Functions and variable scope
- Live coding: To-do app refactor

**Learning objective**

Use the right data structure for the right task, then wrap logic into reusable functions.

## By the end, you can

Choose between `list`, `tuple`, `dict`, `set`, and structure small programs with clean function boundaries.

# Tuples

Immutable sequences and why that matters

# Lists: Quick Refresher

## List properties

- **Mutable** items can change in place
- **Ordered** index-based access
- **Duplicates** allowed
- **Mixed types** are valid

Great when data changes over time.

```
tasks = ["email", "laundry"]
tasks.append("dishes")
tasks[1] = "groceries"
tasks.remove("email")

for task in tasks:
    print(task)
```

Lists are your default dynamic collection.

# Lists vs Tuples

| Feature | List | Tuple |
|---|---|---|
| Mutability | Mutable | Immutable |
| Syntax | `[1, 2, 3]` | `(1, 2, 3)` |
| Typical use | Data that changes | Fixed records and safe keys |

> Choose `list` for evolving state, choose `tuple` for stable structure.

# Tuples in Python

## Core properties

- Immutable after creation
- Ordered and indexable
- Duplicates allowed
- Can hold mixed data types

Useful for coordinates, configs, and return bundles.

```
coords = (10.0, 20.0)

# These fail on purpose:
# coords.append(100.0)
# coords[1] = 10.5
# coords.remove(10.0)
```

**Common misconception**

Tuple methods for resizing do not exist. If structure must change, use a list.

# Tuples in Action: Packing and Unpacking

```
my_tuple = (1, 2, 3, 4, 5)
fruits = ("apple", "banana", "cherry")
mixed = (1, "apple", True)
empty = ()

coordinates = (10, 20)   # packing
x, y = coordinates       # unpacking

a, b = 3, 9
a, b = b, a              # classic tuple swap
```

**Fun Python fact:** `(1)` is just an `int`, while `(1,)` is a one-element tuple.

# Accessing Tuple Elements

```
dice_rolls = (1, 2, 2, 3, 4, 5, 2)

print(dice_rolls[0])      # 1
print(dice_rolls[-1])     # 2
print(dice_rolls[1:3])    # (2, 2)
print(dice_rolls[::-1])   # reversed tuple
```

## Indexing patterns

- `[i]` single element
- `[-1]` last element
- `[a:b]` slicing
- `[::-1]` reverse copy

# Tuple Methods and Operations

```
dice_rolls = (1, 2, 2, 3, 4, 5, 2)

print(len(dice_rolls))
print(dice_rolls.count(2))
print(dice_rolls.index(3))

fake_dice_rolls = (6, 6, 6)
all_dice_rolls = dice_rolls + fake_dice_rolls
print(all_dice_rolls)
```

## Membership checks

`6 in dice_rolls`

`6 not in dice_rolls`

Tuples support the same membership operators as lists and strings.

# Why Immutability Matters

### Data integrity

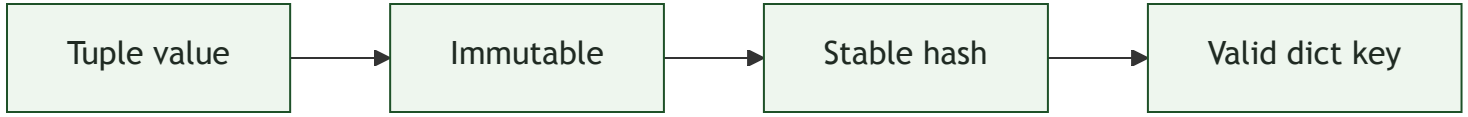
Critical records cannot be changed by accident.

### Performance

Fixed-size memory layout is efficient to store and iterate.

### Hashability

Stable values can become dictionary keys.




Immutability connects tuples directly to dictionary design because keys must remain stable.

# Checkpoint: Tuples

### Q1

What is the type of `(1)`?

A) tuple

B) int

C) list

D) Depends on moon phase

Show Answer

### Q2

What happens with `coords[1] = 99` if `coords` is a tuple?

A) Works and updates index 1

B) TypeError

C) KeyError

D) Only works on Tuesdays

Show Answer

### Q3

Which object can be a dictionary key?

A) `[1, 2]`

B) `(1, 2)`

C) `{"x": 1}`

D) `([1, 2], 1)`

Show Answer

# Dictionaries

Flexible key-value data modeling

# Dictionaries in Python

## Dictionary basics

- Stores `key: value` pairs
- Keys must be unique and immutable
- Values can be any type and can repeat
- Fast lookup by key

```
student_grades = {
    "Alice": 95,
    "Bob": 87,
    "Charlie": 92
}

student = {"name": "Alice", "age": 20}
empty = {}
```

Real-world hook: JSON payloads map directly to dictionaries.

# Key-Value Mental Model

**Keys**

`"name"`

`"age"`

⇄

**Values**

`"Alice"`

`20`

```
student = {
    "name": "Alice",
    "age": 20
}
```

# Keys Must Be Immutable and Unique

```
valid_dict = {
    "name": "Alice",
    100: "ID Number",
    (10, 20): "coordinate",
    "name": "Alicia"   # overwrites previous "name"
}
print(valid_dict)

# invalid_dict = {
#     ["a", "b"]: "list key"   # TypeError
# }
```

**Gotcha:** duplicate keys are not stored twice, the last value wins.

# Accessing Dictionary Values

```
student = {"name": "Alice", "age": 20}

student["name"]        # "Alice"
student["grade"]       # KeyError

student.get("age")     # 20
student.get("grade")   # None
student.get("grade", "N/A")
```

| Method | Missing key behavior |
|---|---|
| `dict[key]` | Raises `KeyError` |
| `dict.get(key)` | Returns `None` or provided default |

# Modifying Dictionaries

```
student = {"name": "Alice", "age": 20}

# add
student["grade"] = "A"

# update
student["age"] = 21

# remove
del student["age"]        # or student.pop("age")

# batch update
student.update({"city": "Budapest"})
```

Use `pop()` when you want both deletion and the removed value.

# Iterating Through Dictionaries

```
person = {"name": "Bob", "age": 30, "city": "New York"}

for key in person:
    print(f"Key: {key}, Value: {person[key]}")

for value in person.values():
    print(f"Value: {value}")

for key, value in person.items():
    print(f"{key} → {value}")
```

Choose `items()` when you need both key and value.

# Dictionary Methods

```
student = {
    "name": "Alice",
    "age": 20,
    "grade": "A"
}

student.keys()
student.values()
student.items()
student.clear()
```

## Notes

- These methods return view objects
- Convert with `list( ... )` when needed
- Views update if dictionary changes

> **Fun Python fact:** dict preserves insertion order (Python 3.7+).

# Membership Operator: `in` and `not in`

## Works with many iterables

list tuple string dict set

For dictionaries, membership checks keys by default.

```
text = "Gandalf the Grey"
print("h" in text)

db_config = {"user": "admin", "password": "012345678"}
print("password" in db_config)         # key lookup
print("admin" in db_config.values())   # value lookup
```

# Checkpoint: Dictionaries

### Q4

Which call avoids `KeyError` for missing keys?

A) `d["x"]`

B) `d.get("x", 0)`

C) `d.keys("x")`

D) `d.unlock("x")`

Show Answer

### Q5

What is printed?

```
d = {"name": "Alice", "name": "Alicia"}
print(d["name"])
```

A) Alice

B) Alicia

C) ["Alice", "Alicia"]

D) Identity crisis

Show Answer

### Q6

Which key is valid in a dictionary?

A) `["a", "b"]`

B) `{"x": 1}`

C) `(10, 20)`

D) `([10], 20)`

Show Answer

# Data Types and Sets

Choosing the right container

# Non-Scalar Data Types

### List

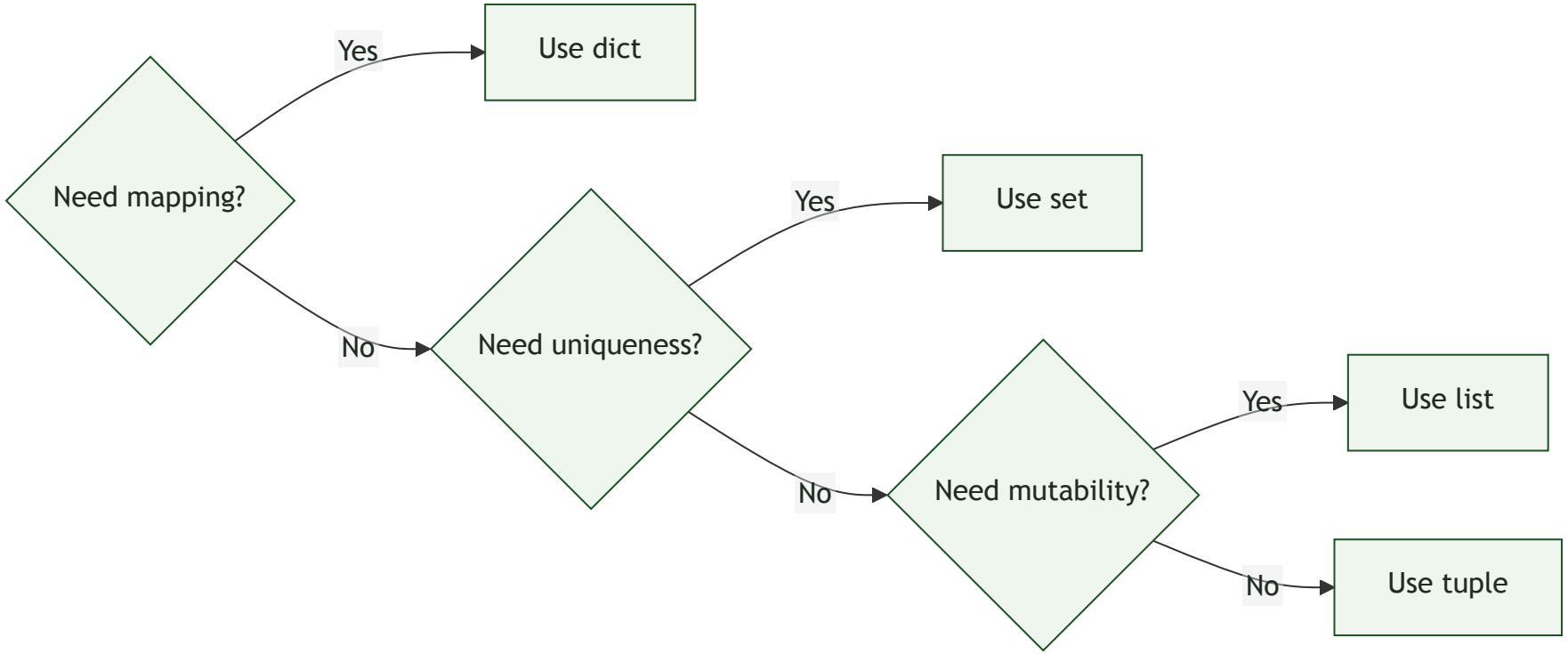
Ordered, mutable sequence

### Tuple

Ordered, immutable sequence

### Dict

Key-value mapping




> Data structure choice should be intentional, not accidental.

# Sets in Python

## Set properties

- Unordered and unique elements
- Mutable collection
- No indexing or slicing
- Excellent for fast membership checks

> **Gotcha:** empty set is `set()`, not `{}`.

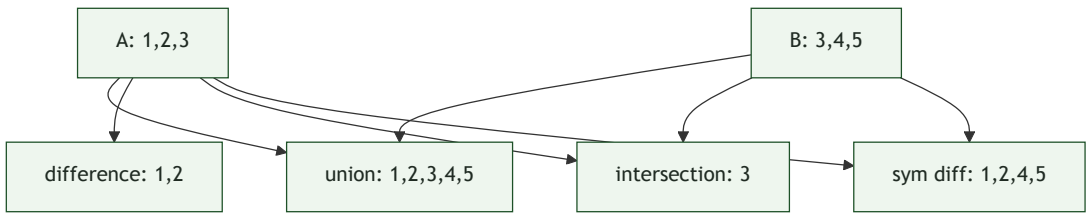
```python
my_set = {1, 2, 3, 4}
print(my_set)

empty_set = set()
my_set.add(5)
my_set.remove(3)
```

# Set Operations

```
a = {1, 2, 3}
b = {3, 4, 5}

print(a.union(b))                # {1,2,3,4,5}
print(a.intersection(b))         # {3}
print(a.difference(b))           # {1,2}
print(a.symmetric_difference(b)) # {1,2,4,5}
```



# Checkpoint: Sets

**Q7**

What does this print?

```
a = {1, 2, 3}
b = {3, 4}
print(a.intersection(b))
```

A) {1, 2, 3, 4}

B) {3}

C) [3]

D) The set is shy and prints nothing

Show Answer

**Q8**

How do you create an empty set?

A) {}

B) []

C) set()

D) summon_set()

Show Answer

# Functions and Scope

Reusable logic and variable visibility

# Functions in Python

## Why functions?

- Follow `DRY` principle
- Improve modularity and readability
- Enable reuse and testing

```python
def function_name(parameter1, parameter2):
    # do something
    return result
```

**Anatomy:** `def` + name + parameters + indented body + optional return.

# Calling a Function

```python
# 1) define
def greet():
    print("Hello, world!")
    print("Welcome to the world of functions.")

# 2) call
greet()
greet()
```

Define once, call many times.

# Parameters vs Arguments

## Definitions

- **Parameter:** variable in function definition
- **Argument:** concrete value passed on call

```python
def greet_person(name):   # parameter
    print(f"Hello, {name}!")

greet_person("Gandalf")   # argument
greet_person("Luke")
```

# Return Values

```python
def add(a, b):
    result = a + b
    return result

sum_result = add(5, 3)
print(sum_result)  # 8
```

**No return**

Implicitly returns `None`

**Multiple values**

Returned as a tuple

**Early return**

Exits function immediately

# Positional and Keyword Arguments

```
def describe_pet(pet_name, animal_type):
    print(f"I have a {animal_type} named {pet_name}.")

describe_pet("Willie", "hamster")                  # positional
describe_pet(animal_type="cat", pet_name="Milo")  # keyword
```

Keyword arguments improve readability when function signatures grow.

# Default Arguments

```python
def describe_pet(pet_name, animal_type="dog"):
    print(f"I have a {animal_type} named {pet_name}.")

describe_pet("Fido")
```

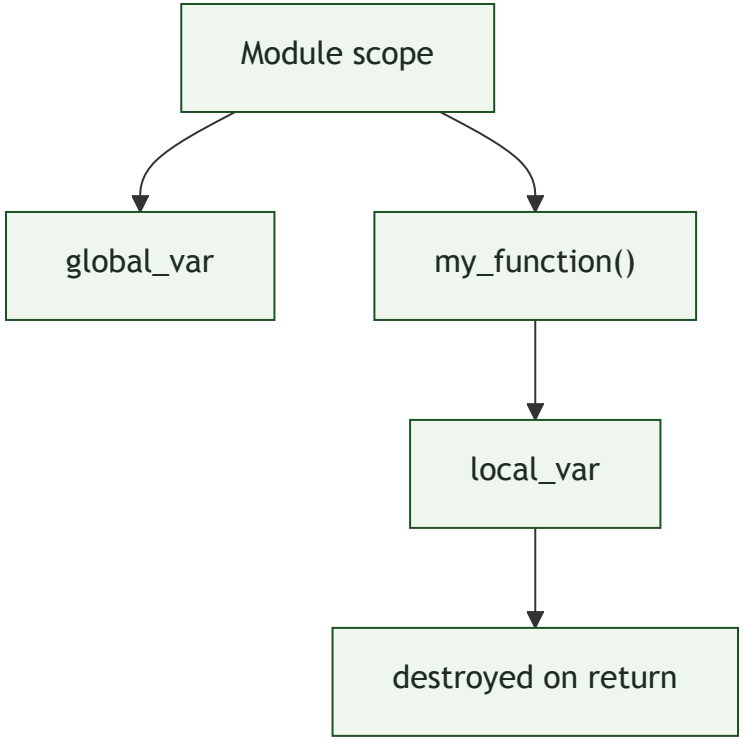
**Gotcha:** avoid mutable defaults.

```python
# bad
def add_item(item, items=[]):
    items.append(item)
    return items

# better
def add_item(item, items=None):
    if items is None:
        items = []
    items.append(item)
    return items
```

# Variable Scope: Local vs Global

- **Local scope:** inside a function
- **Global scope:** module-level variables
- Functions can read globals
- Locals are created/destroyed on call



# Scope Example

```
global_var = "I am global"

def my_function():
    local_var = "I am local"
    print(global_var)
    print(local_var)

my_function()
print(global_var)
# print(local_var)  # NameError
```

Local names are invisible outside the function.

# No Block Scope in Python

```python
for i in range(3):
    loop_var = 100
    print(f"Inside loop: i={i}, loop_var={loop_var}")

print(i)         # 2
print(loop_var)  # 100
```

Unlike Java/C++, loop variables remain available after the loop ends.

# Checkpoint: Functions and Scope

### Q9

What is printed?

```
def f(x=5):
    return x * 2
print(f())
```

A) 5

B) 10

C) None

D) Server overheats

Show Answer

### Q10

Why can mutable default arguments be risky?

A) They are recreated on every call

B) They are shared across calls

C) Python forbids them

D) They require admin rights

Show Answer

# Live Coding: To-do App

Refactoring with dictionaries and functions

# Iteration Goal

### From Iteration 5

Simple loop with list-based tasks and basic commands.

### Now in Iteration 6/7

Add richer task structure ( `id` , `status` ) and split logic into clean functions.

# Feature 6.2: Task Metadata

```
task = {
    "id": 101,
    "title": "finish lab",
    "status": "pending"
}

ALLOWED_STATUS = {"pending", "completed"}
```

Store tasks as dictionaries so fields stay explicit and extensible.

# Refactor Principles

### Separate I/O

Logic functions return data, main loop handles input/output.

### Single responsibility

One function, one job: add, find, complete, render.

### Validate input

Never trust user input; check IDs and statuses.

> Return data structures, then render messages in one small formatting layer.

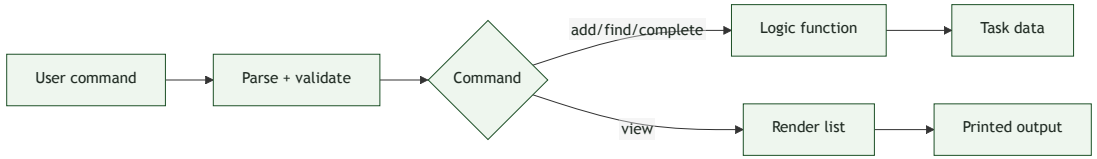
# Iteration 7: New Commands

## Feature 7.1

`find <id>` searches by task ID.

## Feature 7.2

`complete <id>` updates task status.



# Summary

## Tuples

- Immutable sequence
- Good for fixed records
- Useful as dictionary keys

## Dictionaries and Sets

- Dict: key-value lookup
- Set: uniqueness + operations
- Membership checks are central

## Functions and Scope

- Reusable logic blocks
- Parameter/argument discipline
- Understand local vs global names

Next step: combine data structures + functions to build maintainable mini applications.

# Thank You for Your Attention!

*"Simple is better than complex."*

-- The Zen of Python

tuples dicts sets functions